

## CSCE 645 GEOMETRIC MODELING · MODULE 03 · LECTURE NOTES

# Curves: Bézier and Continuity

*The one-dimensional case, where every idea is visible.*

## MODULE OUTCOMES

- Evaluate Bézier curves by Bernstein basis and by de Casteljau.
- State and use the convex hull, affine invariance, and variation diminishing properties.
- Subdivide a curve and use it for adaptive tessellation.
- Distinguish parametric from geometric continuity and know which matters when.
- Explain why high-degree Bézier curves are impractical.

## 1 Definition

$$\mathbf{B}(t) = \sum_{i=0}^n \mathbf{b}_{i,n}(t) \mathbf{P}_i, \quad \mathbf{b}_{i,n}(t) = \mathbf{C}(n,i) t^i (1-t)^{n-i}$$

The weights  $\mathbf{b}_{i,n}$  are the **Bernstein polynomials** of degree  $n$ . Two facts about them generate everything else:

- They are non-negative on  $[0,1]$ , since  $t$  and  $(1-t)$  both are.
- They sum to 1 for every  $t$  — immediately, by the binomial theorem applied to  $(t + (1-t))^n = 1$ .

## THE CURVE IS A CONVEX COMBINATION

Non-negative weights summing to one is precisely the definition of a convex combination, and CSCE 641 Module 02 noted that such a combination of *points* is well defined where a plain sum is not.

Every property in §2 is a consequence of this one observation. Treating them as a list to memorise obscures that they are all the same fact seen from different angles.

### 1.1 de Casteljau's algorithm

```
vec3 de_casteljau(std::vector<vec3> P, float t) {
    while (P.size() > 1) {
        for (size_t i = 0; i + 1 < P.size(); ++i)
            P[i] = (1 - t) * P[i] + t * P[i + 1];
        P.pop_back();
    }
    return P[0];
}
```

Repeated linear interpolation: interpolate between adjacent control points, then between the results, until one point remains. It is  $O(n^2)$  rather than the  $O(n)$  of Horner evaluation, and it is preferred anyway for two reasons.

- **Numerical stability.** Every intermediate value is a convex combination of points on the curve's hull, so no quantity grows. Direct evaluation of the Bernstein sum involves large binomial coefficients multiplying small powers, with substantial cancellation at high degree.
- **It produces the subdivision for free (§3),** which is what you actually want for rendering.

## 2 Properties

| Property                      | Statement                                                                                | Consequence                                                                                                                       |
|-------------------------------|------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| <b>Endpoint interpolation</b> | $B(0) = P_0$ and $B(1) = P_n$ .                                                          | Segments can be joined by sharing endpoints; the interior control points pull without being touched.                              |
| <b>Convex hull</b>            | The curve lies entirely within the convex hull of its control points.                    | A bounding volume for free, without evaluating the curve. Makes conservative culling and the flatness test of §3 valid.           |
| <b>Affine invariance</b>      | Applying an affine map to the control points produces exactly the transformed curve.     | Transform $n$ points rather than a thousand samples.                                                                              |
| <b>Variation diminishing</b>  | No straight line intersects the curve more times than it intersects the control polygon. | The curve cannot oscillate more than the polygon you drew — which is why Bézier curves feel predictable to designers.             |
| <b>End tangents</b>           | $B'(0) = n(P_1 - P_0)$ , $B'(1) = n(P_n - P_{n-1})$ .                                    | The tangent direction at a join is controlled directly by the adjacent control point — the basis of continuity constraints in §4. |

### AFFINE INVARIANCE AND ITS LIMIT

Transforming the four control points of a cubic costs four matrix multiplies, and the resulting curve is exactly the transform of the original. No resampling is required.

This holds for affine transformations — translation, rotation, scale, shear. **It fails** for perspective projection, because perspective involves a division and is therefore not affine.

The repair is to use *rational* curves, where each control point carries a weight and the curve is a ratio of polynomials. These are invariant under projective transformations as well, and they are the reason NURBS exist. Module 04.

## 3 Subdivision and rendering

Running de Casteljau at parameter  $t$  and retaining the intermediate values produces the control polygons of the two halves:

- The **first** point computed at each level, taken in order, is the control polygon of the curve on  $[0, t]$ .
- The **last** point at each level, taken in reverse order, is the control polygon of the curve on  $[t, 1]$ .

Both halves are exact Bézier curves of the same degree — nothing is approximated. This gives the standard rendering algorithm:

```
void tessellate(const Bezier& c, float tol, std::vector<vec3>& out) {
    if (flat_enough(c, tol)) { out.push_back(c.P[3]); return; }
    auto [L, R] = subdivide(c, 0.5f);
    tessellate(L, tol, out);
    tessellate(R, tol, out);
}

// flat_enough: maximum distance from the interior control points
// to the chord P0-P3, compared against tol.
```

### WHY THE FLATNESS TEST IS CORRECT AND NOT JUST PLAUSIBLE

The convex hull property guarantees the curve lies within the hull of its control points. If all control points are within  $tol$  of the chord, the entire curve is too.

So the test is **conservative**: it may subdivide more than strictly necessary, and it can never under-tessellate. That is exactly the guarantee you want from a renderer, and it depends on a property that would otherwise look like trivia.

Adaptive subdivision also naturally produces more segments where curvature is high and fewer where the curve is nearly straight, which is the right distribution of effort.

## 4 Continuity

| Order | Parametric $C^n$                                                                                                                                | Geometric $G^n$                                                                                |
|-------|-------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| 0     | Positions match.                                                                                                                                | Identical to $C^0$ .                                                                           |
| 1     | First derivatives match <i>as vectors</i> : same direction <b>and</b> magnitude. The curve is traversed at a consistent speed through the join. | Tangent <i>directions</i> match; magnitudes may differ. The join looks smooth; speed may jump. |
| 2     | Second derivatives match: acceleration is continuous.                                                                                           | Curvature matches. Reflections flow smoothly across the join.                                  |

$C^n$  implies  $G^n$ , but not the reverse. A curve can be  $G^1$  and not  $C^1$ : the two segments meet with the same tangent direction but different tangent magnitudes.

#### WHICH YOU NEED DEPENDS ON WHAT IS MOVING

**Something travels along the curve at a parameter-driven rate** — a camera on a spline path, an animated object, a CNC cutting head. You need  $C^1$ . A  $G^1$  join is visually smooth and produces a sudden change in speed, which reads as a jolt and is surprisingly noticeable.

**The curve is purely a shape** — a font outline, a silhouette, a profile to be swept.  $G^1$  is sufficient, and it gives the designer an extra degree of freedom per join.

**The resulting surface will be reflective.** You need  $G^2$ . A curvature discontinuity is invisible in the silhouette and unmistakable in a reflection, appearing as a crease in the reflected environment. This is why automotive surfacing standards are stated in terms of  $G^2$  and why 'zebra stripe' analysis exists.

### 4.1 Joining two cubics

For two cubic segments with control points  $P_0 \dots P_3$  and  $Q_0 \dots Q_3$ :

- $C^0$ :  $Q_0 = P_3$ .
- $C^1$ : additionally  $Q_1 - Q_0 = P_3 - P_2$  — the three points  $P_2, P_3, Q_1$  are collinear and evenly spaced.
- $G^1$ :  $P_2, P_3, Q_1$  collinear, with spacing unconstrained.
- $C^2$ : a further condition on  $Q_2$ , which uses up most of the second segment's freedom — which is exactly the problem B-splines solve by enforcing it automatically.

## 5 Why not just raise the degree?

A Bézier curve's degree is tied to its control point count. Wanting more control means raising the degree, and that goes badly in three ways.

- 1 **Global control.** Every Bernstein polynomial is non-zero throughout  $(0,1)$ , so every control point influences every point of the curve. Moving one point to fix a local problem changes the whole curve, which makes editing a long curve impractical.
- 2 **Numerical conditioning.** High-degree Bernstein coefficients become large while the basis functions become small, and direct evaluation suffers cancellation. de Casteljau helps but the underlying conditioning is still poor.
- 3 **Oscillation.** High-degree polynomials tend to wiggle between their constraints — the same phenomenon as Runge's oscillation in polynomial interpolation.

The answer is to use many **low-degree segments** joined with explicit continuity constraints. Cubics are the near-universal choice: degree 3 provides enough freedom to achieve  $C^2$  joins while remaining numerically well behaved.

Managing those constraints by hand across dozens of segments is tedious and error-prone. Module 04 introduces B-splines, which are exactly this construction with the continuity conditions built into the basis so that they hold automatically.

## Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

| Resource                                                                | What it covers for this module                                                                                                    |
|-------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">Keenan Crane — DDG, the curves material</a>                 | Bézier curves with the properties derived rather than asserted.                                                                   |
| <a href="#">Freya Holmér — The Beauty of Bézier Curves (free video)</a> | The clearest visual explanation of de Casteljau and the control properties available anywhere. Watch before reading.              |
| <a href="#">A Primer on Bézier Curves (free, interactive)</a>           | Comprehensive, interactive, with derivations and working code for everything in this module including subdivision and offsetting. |
| <a href="#">GAMES101 Lecture 11 — Geometry 1 (Curves)</a>               | The condensed graphics-oriented treatment you met in CSCE 641.                                                                    |

## Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement Bézier evaluation both by direct Bernstein summation and by de Casteljau. Compare results at degree 3, 10, and 30, and quantify the numerical divergence.
- 2 Implement subdivision at arbitrary  $t$ . Verify that evaluating the left half at parameter  $s$  gives the same point as evaluating the original at  $t \cdot s$ .
- 3 Implement adaptive flatness-based tessellation. Render a curve at three tolerances and report the segment counts; confirm that segments cluster where curvature is high.
- 4 Verify affine invariance numerically: transform the control points, evaluate, and compare against transforming evaluated points. Then do the same with a perspective matrix and show that it fails.
- 5 Construct two cubic segments joined  $G^1$  but not  $C^1$ . Animate a point along the pair at constant parameter rate and make the speed discontinuity visible by plotting arc length against  $t$ .
- 6 Construct a join that is  $G^1$  but not  $G^2$ . Sweep a surface along it, render with a mirror material and an environment map, and capture the crease in the reflection.
- 7 Fit a single Bézier curve of degree 15 to a set of points and observe the oscillation. Then fit a chain of cubics and compare.

## Self-check

---

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What two facts about the Bernstein basis make a Bézier curve a convex combination, and why does that matter?
- 2 Give two reasons to prefer de Casteljau over direct evaluation.
- 3 Why is the convex hull property what makes flatness-based tessellation correct?
- 4 What does affine invariance buy, and why does it fail under perspective?
- 5 Distinguish  $C^1$  from  $G^1$  and give a situation where each is the one you need.
- 6 Why does  $C^2$  matter for reflective surfaces but not for silhouettes?
- 7 Give three reasons high-degree Bézier curves are impractical.